

La Cancha Inteligente

Desafío 3 - Copa de Algoritmia y Programación UADE 2026

Carreras de Informatica y Sistemas

Integrantes

Lucas Abad

Thiago Albornoz

Valentino Sarniguette

Gaston Trezeguet

Valentin Zaccari

1. Descripción general

El proyecto La Cancha Inteligente consiste en un simulador táctico desarrollado en Python. El programa representa una cancha de futbol mediante una matriz de 100 filas por 60 columnas, donde se pueden cargar jugadores, obstáculos y distintas situaciones de juego.

El sistema permite analizar una jugada a partir de la ubicación de los jugadores en la cancha. Entre sus funcionalidades principales se encuentran el movimiento de jugadores, el calculo de distancia a la pelota, la detección de pases posibles y la verificación de camino libre al arco.

Además de cumplir con los requerimientos principales del desafío, el programa incorpora un menú interactivo, validaciones de datos, escenario de prueba automático, reinicio de cancha y borrado de jugadores.

2. Requisitos

Para ejecutar el programa se necesita Python 3 instalado. No se requieren librerías externas. El programa utiliza estructuras básicas de Python: listas, diccionarios, funciones, condicionales y ciclos.

3. Cómo ejecutar el programa

El archivo principal debe ejecutarse desde una terminal o desde un entorno como Visual Studio Code.

```
python desafio3.py
```

4. Estructura general del programa

El programa esta organizado en bloques funcionales para que cada parte tenga una responsabilidad clara.

Bloque	Responsabilidad
1	Constantes y creacion de la cancha
2	Validaciones basicas
3	Creacion y posicionamiento de jugadores
4	Obstáculos
5	Movimiento de jugadores
6	Distancia a la pelota
7	Pases posibles
8	Camino libre al arco
9	Funciones de ingreso de datos
10	Opciones del submenu
11	Escenario de prueba
12	Menu
13	Inicio del programa

5. Representación de la cancha

La cancha se representa mediante una matriz de 100 x 60. Las filas válidas van de 0 a 99 y las columnas válidas van de 0 a 59.

Simbolo	Significado
.	Celda vacia
A	Jugador de Argentina
B	Jugador de Brasil
X	Obstáculo

6. Representación de los jugadores

Cada jugador se representa mediante un diccionario de Python. Los jugadores se almacenan dentro de un diccionario general llamado jugadores, donde cada clave es un identificador numérico.

```
{  
    "nombre": "Messi",  
    "equipo": "A",  
    "fila": 50,  
    "columna": 20,  
    "rol": "delantero",  
    "tiene_pelota": "S"  
}
```

7. Equipos y roles validos

Equipo	Significado
A	Argentina
B	Brasil

Los roles validos son arquero, defensor, mediocampista y delantero. Si el usuario ingresa un equipo o rol invalido, el programa muestra un mensaje de error y vuelve a pedir el dato.

8. Funcionalidades principales

8.1 Crear cancha

La función `crear_cancha()` genera una matriz de 100 filas y 60 columnas. Cada celda se inicializa con un punto.

8.2 Agregar jugadores

Permite cargar nombre, equipo, fila, columna, rol y posesión de pelota. Valida equipo, rol, posición, celda ocupada y pelota única.

8.3 Agregar obstáculos

Permite colocar un obstáculo X en una celda libre dentro de los límites de la cancha.

8.4 Mover jugadores

Permite mover un jugador arriba, abajo, izquierda o derecha. Valida dirección, límites y celda destino libre.

8.5 Calcular distancia a la pelota

Calcula la distancia Manhattan entre cada jugador y el portador de la pelota.

```
abs(fila_jugador - fila_pelota) + abs(columna_jugador - columna_pelota)
```

8.6 Detectar pases posibles

Analiza si el jugador con pelota puede pasar a un compañero en la misma fila o columna, sin rivales ni obstáculos entre ellos.

8.7 Detectar camino libre al arco

Analiza si un delantero está en mitad ofensiva y no tiene rivales u obstáculos en su misma fila hacia el arco rival.

8.8 Listar jugadores

Muestra ID, nombre, equipo, fila, columna, rol y posesión de pelota.

8.9 Borrar jugador

Elimina un jugador por ID, libera su celda y avisa si tenía pelota.

8.10 Reiniciar cancha

Crea una cancha nueva y elimina jugadores y obstáculos cargados.

8.11 Mostrar cancha

Imprime la matriz completa, avisando previamente que la salida es extensa.

8.12 Escenario de prueba automático

Carga una jugada predefinida para evaluar rápidamente distancias, pases y camino al arco.

9. Menú del sistema

Menú principal:

```
===== MENU PRINCIPAL =====
```

- 1 - Modo de uso
- 2 - Ejecutar cancha inteligente
- 3 - Acerca de
- 0 - Salir

Submenú del simulador:

```
===== SIMULADOR DE CANCHA =====
```

- 1 - Agregar jugador
- 2 - Agregar obstaculo
- 3 - Mover jugador
- 4 - Analizar jugada
- 5 - Listar jugadores
- 6 - Mostrar cancha
- 7 - Cargar escenario de prueba
- 8 - Reiniciar cancha
- 9 - Borrar jugador
- 0 - Volver

10. Validaciones implementadas

- Equipo valido: solo A o B.
- Rol valido: arquero, defensor, mediocampista o delantero.
- Posición dentro de la cancha.
- Celda libre antes de agregar jugador u obstáculo.
- Una sola pelota en juego.
- Dirección valida para movimiento.
- Ingreso numérico controlado con try/except.

- Nombre de jugador no vacío.
- Cancelación de carga escribiendo NO.
- Bloqueo de movimiento fuera de cancha o hacia celda ocupada.
- Control de análisis si no hay suficientes jugadores o no hay pelota.
- Liberación de celda al borrar un jugador.

11. Casos de prueba recomendados

Caso	Accion	Resultado esperado
1	Cargar escenario de prueba	Jugadores y obstáculos listos para analizar
2	Agregar jugador valido	Se agrega y se marca en la cancha
3	Superponer jugador	Se rechaza celda ocupada
4	Duplicar pelota	Se evita segunda pelota
5	Mover fuera de cancha	Movimiento bloqueado
6	Mover a celda ocupada	Movimiento bloqueado
7	Agregar obstaculo	Celda marcada con X
8	Pase bloqueado	Se informa bloqueo
9	Pase posible	Se informa pase disponible
10	Camino bloqueado	Delantero sin camino libre
11	Borrar jugador	Se libera la celda
12	Reiniciar cancha	Cancha vuelve a estar vacia

Caso de prueba rápido recomendado:

- 2 - Ejecutar cancha inteligente
- 7 - Cargar escenario de prueba
- 5 - Listar jugadores
- 4 - Analizar jugada
- 9 - Borrar jugador
- 8 - Reiniciar cancha
- 0 - Volver
- 0 - Salir

12. Decisiones de diseño

Uso de matriz

Se eligió representar la cancha como una matriz porque la consigna trabaja con posiciones de fila y columna.

Uso de diccionarios para jugadores

Cada jugador se representa como diccionario. Además, un diccionario general con ID numérico facilita la selección desde el menú.

Pelota como atributo del jugador

La pelota se representa con el campo tiene_pelota y no como celda independiente.

Menú interactivo

El menú mejora la experiencia del usuario y facilita la evaluación manual.

Escenario de prueba

Permite verificar rápidamente las funcionalidades principales sin cargar datos manualmente.

Borrado y reinicio

Permiten corregir errores de carga y probar nuevas jugadas de manera cómoda.

13. Restricciones y aclaraciones

El programa no simula un partido completo. Representa una situación puntual de juego sobre una cancha matricial.

El sistema detecta pases posibles, pero no ejecuta el pase ni cambia la posesión de la pelota. Esta decisión se tomo porque el desafío solicita detectar si existen pases posibles, no simular una acción completa de pase.

La pelota no ocupa una celda propia en la matriz. Esta asociada al jugador que la posee.

14. Conclusión

El programa desarrollado cumple con los requerimientos principales del Desafío 3 mediante una solución modular, validada e interactiva.

La cancha se representa mediante una matriz de 100 x 60, los jugadores mediante diccionarios y las funcionalidades se organizan en funciones específicas. El sistema permite cargar y mover jugadores, agregar obstáculos, calcular distancias, detectar pases posibles y analizar camino libre al arco.

Además, se agregaron mejoras de experiencia de usuario como menú interactivo, escenario de prueba, reinicio de cancha y borrado de jugadores. Estas mejoras permiten una evaluación mas clara y facilitan el uso del sistema sin modificar el código fuente.